

LEVEL 4

Chapter 5: Database Deep Dive

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 5: Database Deep Dive

← [Chapter 4: Core Building Blocks](#) · [Index](#) · Next → [Chapter 6: Distributed Systems](#)

5.1 Indexes — Why Reads Get Fast

Simple explanation: Without an index, finding a row means scanning every row in the table (a "full table scan") — like reading every page of a book to find one sentence. An index is a separate, sorted data structure that lets the database jump almost directly to the matching rows — like using the index at the back of the book.

Primary key: the column (or set of columns) that uniquely identifies each row; the database automatically builds an index on it.

Secondary index: any additional index you build on other columns to speed up queries that filter/sort by them (e.g., indexing `email` on a `users` table so login lookups don't scan everything).

B-Tree index: the default index structure in almost every relational database (MySQL InnoDB, PostgreSQL). A balanced tree structure where each lookup, insert, or range query takes $O(\log n)$ time, and — critically — it keeps data **sorted**, which means it's efficient for range queries (`WHERE created_at BETWEEN x AND y`) and sorting (`ORDER BY`), not just exact-match lookups.

Hash index: maps a key directly to a location via a hash function — $O(1)$ average lookup for exact matches, but **cannot** do range queries or sorting (a hash of "5" and a hash of "6" have no relationship, so "give me everything between 5 and 6" is meaningless to a hash index). Redis and some NoSQL stores use hash indexing internally for this reason; PostgreSQL supports hash indexes for pure equality lookups, but B-Tree is the sane default for almost everything.

Clustered vs. non-clustered index:

	Clustered index	Non-clustered (secondary) index
What it is	The table's actual row data is physically stored in this index's order	A separate structure that stores the indexed column(s) + a pointer/reference back to the actual row
How many per table	Exactly one (usually the primary key)	As many as you want
Lookup cost	Direct — you're already at the data	Extra hop: find the pointer, then fetch the actual row ("bookmark lookup")
Example		

	Clustered index	Non-clustered (secondary) index
	InnoDB (MySQL) always clusters on the primary key	Any <code>CREATE INDEX</code> on a non-primary-key column

Interview question: "You added an index on `orders.user_id` but a query filtering on it is still slow. Why?"

Ideal senior answer: "A few usual suspects: the query might be selecting columns not covered by the index, forcing a bookmark lookup back to the full row for every match — a covering index (one that includes all needed columns) avoids that extra hop. Or the index might not be selective enough — if 90% of rows share the same `user_id` value (unlikely here, but the general principle), the optimizer may reasonably choose a full scan over the index anyway. Or there's an implicit type conversion in the WHERE clause silently disabling index use. I'd run `EXPLAIN` / `EXPLAIN ANALYZE` first rather than guess."

Composite (multi-column) indexes: an index on `(user_id, created_at)` is efficient for queries filtering on `user_id` alone, or `user_id` + `created_at` together — but *not* efficient for filtering on `created_at` alone, because a composite index is sorted by its leftmost column first (the "leftmost prefix rule"). Getting the column order right, based on actual query patterns, is a real interview-worthy detail.

Query optimization / joins basics: the database's query planner decides *how* to execute your SQL (which indexes to use, join order, join algorithm). Know that joins across very large tables are expensive, and at high scale you often deliberately **denormalize** (duplicate some data to avoid a join) to trade storage for query speed — a classic, explicit HLD trade-off worth naming out loud.

5.2 Transactions, Isolation, and Locking

ACID, precisely:

- **Atomicity:** a transaction's operations all happen, or none do — no partial writes.
- **Consistency:** a transaction moves the database from one valid state to another, respecting all constraints (foreign keys, uniqueness).
- **Isolation:** concurrent transactions don't interfere with each other's intermediate states.
- **Durability:** once committed, the write survives a crash.

Isolation levels — the interview-favorite table

Level	Prevents	Allows
Read Uncommitted	Nothing	Dirty reads (seeing another transaction's uncommitted changes)

Level	Prevents	Allows
Read Committed	Dirty reads	Non-repeatable reads (re-reading the same row twice in one transaction gives different results, because another transaction committed a change in between)
Repeatable Read	Dirty reads, non-repeatable reads	Phantom reads (re-running the same range query returns <i>new rows</i> that another transaction inserted) — though MySQL's InnoDB Repeatable Read actually prevents most phantoms too via gap locks/MVCC
Serializable	All of the above	Nothing — behaves as if transactions ran one at a time, strictly sequentially

Trade-off: stronger isolation = more correctness guarantees but more locking/blocking and lower throughput. **Read Committed is PostgreSQL's default; Repeatable Read is MySQL InnoDB's default** — a genuinely useful fact to know cold.

Locks: a transaction acquires a lock on a row (or table, or range) it's reading/writing to prevent conflicting concurrent access. **Shared locks** (multiple readers okay) vs. **exclusive locks** (only one writer, blocks everyone else).

Deadlocks: Transaction A holds a lock on row 1 and wants row 2; Transaction B holds a lock on row 2 and wants row 1 — neither can proceed. Databases detect this (a wait-for graph cycle) and forcibly abort one transaction (the "deadlock victim") so the other can continue. **Application-level fix:** always acquire locks (or update rows) in a consistent, agreed-upon order across your codebase — this alone eliminates most deadlocks in practice.

MVCC (Multi-Version Concurrency Control): the mechanism most modern databases (PostgreSQL, MySQL InnoDB) use to achieve high concurrency *without* readers blocking writers. Instead of locking a row for reads, the database keeps multiple versions of a row and gives each transaction a consistent "snapshot" view as of when it started — readers see the version that was current when their transaction began, writers create a new version rather than blocking to overwrite. This is *why* "readers never block writers, writers never block readers" is true in Postgres/InnoDB — a genuinely good interview fact.

Interview question: "A **SELECT** and an **UPDATE** are running concurrently on the same row. Does the SELECT block?"

Ideal senior answer: "In PostgreSQL or MySQL InnoDB, no — thanks to MVCC, the SELECT sees a consistent snapshot as of when its transaction started, and doesn't wait on the UPDATE's lock. The UPDATE still needs an exclusive lock to actually write, so a second concurrent UPDATE on the same row would block, but reads are non-blocking by design. This is a big part of why these databases handle read-heavy concurrent workloads well without needing to route every read through a cache just to avoid lock contention."

5.3 Replication

Simple explanation: keeping copies of the same data on multiple machines — for durability (survive a machine dying), availability (serve reads even if one node is down), and scaling reads (spread read traffic across replicas).

Topology	How it works	Trade-off
Leader-follower (primary-replica)	All writes go to one leader; the leader streams changes to N followers; reads can be served from followers	Simple, strong consistency for writes; a single write bottleneck, and a failover process is needed if the leader dies
Multi-leader	Multiple nodes accept writes (often one per region/datacenter), replicating to each other	Lower write latency per region, survives a whole region going down without stopping writes; but conflicting concurrent writes to the same record need conflict resolution logic
Leaderless	Any node can accept a read or write; the client (or a coordinator) writes/reads from multiple nodes and uses quorums to reconcile (Cassandra, DynamoDB's underlying model)	Very high availability, no single point of failure for writes; consistency is probabilistic/tunable via quorum settings, and conflict resolution (last-write-wins, vector clocks) is a real design concern

Read-after-write consistency: a guarantee that a user who just wrote data will see their own write on the next read — even if that read happens to hit a replica that hasn't caught up yet. Commonly solved by routing a user's own reads to the leader for a short window after they write, or by "read your own writes" session stickiness.

Eventual vs. strong consistency, precisely: **strong consistency** means any read, from any node, after a write completes, sees that write immediately. **Eventual consistency** means if no new writes occur, all replicas will *eventually* converge to the same value — but there's a window where different nodes can return different answers. Choosing between them is a direct CAP/PACELC trade-off (Chapter 2) applied to a specific piece of data, not a global, once-and-for-all decision — real systems mix both, deliberately, for different data.

Interview question: "A user updates their profile picture and immediately refreshes the page but still sees the old one. What happened, and how do you fix it?"

Ideal senior answer: "Classic replication lag — the write went to the leader, but their subsequent read got routed to a follower that hasn't caught up yet. Fixes, roughly in order of how much complexity I'd accept: route that user's reads to the leader for a short window right after their own write ('read-your-writes' consistency); or have the client optimistically update the UI immediately from the write response instead of re-fetching; or, if using a CDN/cache in front of images specifically, this is also a cache-invalidation problem layered on top, which I'd solve with the versioned-URL trick from Chapter 4."

5.4 Sharding and Partitioning

Vertical partitioning: splitting a table by *columns* — e.g., moving rarely-accessed large text/blob columns into a separate table, keeping the frequently-queried "hot" columns compact. Reduces I/O for common queries.

Horizontal partitioning (sharding): splitting a table by *rows* — each shard holds a subset of rows, typically on a completely separate database instance. This is what people usually mean by "sharding," and it's the step you take when a single machine can no longer hold the data or handle the write throughput, even after vertical scaling and read replicas.

Sharding strategies:

Strategy	How	Risk
Range-based	Shard by a range of the key (users A-M on shard 1, N-Z on shard 2)	Easy to reason about, but prone to hot spots if data/traffic isn't evenly distributed across ranges (e.g., alphabetical name distribution isn't uniform)
Hash-based	Hash the shard key, use the hash to pick a shard	Much better distribution, but range queries ("give me all orders from March") now have to fan out across every shard
Consistent hashing	Hash-based, but nodes and keys are placed on a conceptual ring; adding/removing a node only remaps a small fraction of keys instead of nearly all of them	The standard approach in distributed caches (Redis Cluster) and databases (Cassandra, DynamoDB) — deep dive below
	A separate service maps each key (or key range) to its shard explicitly	Maximum flexibility for rebalancing, but the lookup service itself becomes a

Strategy	How	Risk
Directory-based (lookup table)		critical, must-be-highly-available dependency

Consistent Hashing — worth understanding, not just naming

The problem it solves: with naive hashing ($\text{shard} = \text{hash}(\text{key}) \% N$), adding or removing a single node changes N , which reshuffles *almost every key's* target shard — a catastrophic amount of data movement for what should be a routine capacity change.

How it works: imagine a circle (the "hash ring") representing all possible hash values, from 0 to some max, wrapping back to 0. Both servers and keys are hashed onto this same ring. A key is "owned" by the first server found walking clockwise from the key's position. When you add a server, it only takes over the keys between itself and the *previous* server on the ring — everyone else's assignment is untouched. When a server is removed, only its keys move to the next server clockwise — again, everyone else is untouched.

Virtual nodes: in practice, each physical server is placed on the ring multiple times (as many "virtual nodes"), which smooths out uneven load distribution that plain consistent hashing can otherwise produce with a small number of real nodes.

Hot partitions: even with good key distribution overall, one *specific* key (a viral post, a major merchant's transaction stream) can dominate a single shard's traffic regardless of how evenly the hashing algorithm distributed everything else — this is the sharding-layer version of the "hot key" problem from Chapter 4's caching section, and the fixes rhyme: split the hot entity across sub-keys, or give it dedicated capacity.

Rebalancing: the process of moving data between shards when capacity changes — consistent hashing minimizes how much data needs to move, but "minimizes" isn't "zero," and rebalancing at scale is a genuinely operationally delicate process (usually done gradually, with dual-write or shadow-read verification periods) that's worth mentioning you'd plan for explicitly rather than treat as instantaneous.

Interview question: "Why is consistent hashing specifically preferred over $\text{hash}(\text{key}) \% N$ for a distributed cache?"

Ideal senior answer: "Because $\% N$ ties every key's location to the *current* node count — scaling from 10 to 11 nodes remaps roughly 90% of keys, which for a cache means a near-total cache wipe and a stampede of misses hitting the database simultaneously. Consistent hashing bounds the remapped fraction to roughly $1/N$ of keys when you add one node, which is the whole point — it makes horizontal scaling of a distributed cache or database operationally survivable."

5.5 The Database Scaling Ladder

This is the step-by-step progression interviewers expect you to reason through explicitly, not jump straight to the end of:

```
1 single database
  ↓ (read traffic growing)
add read replica(s)
  ↓ (read traffic still growing, same hot data repeatedly read)
add a cache in front (Chapter 4)
  ↓ (write traffic or data volume outgrows one machine)
partition/shard the data
  ↓ (need this to work across regions / need extreme write availability)
move to a distributed database designed for this from the ground up
(Cassandra, DynamoDB, CockroachDB, Spanner)
```

Why the order matters in an interview: jumping straight to "I'll shard the database" for a system that's nowhere near a single machine's capacity is a real red flag — it signals reaching for complexity before it's earned, exactly the "no trade-offs" mistake pattern in Chapter 23. The strong answer is always: "here's the next cheapest lever, here's when I'd know I need the next one" — i.e., name the *signal* that would tell you to move to the next rung (e.g., "once write throughput approaches what a single primary can sustain, or storage exceeds what fits comfortably on one instance, that's when I'd shard rather than before").

Chapter 5 Interview Drill

1. Explain why MVCC means readers don't block writers, in your own words.
2. State the default isolation level for PostgreSQL and for MySQL InnoDB, and what each one still allows to happen.
3. Walk through consistent hashing's advantage over `hash % N`, using the "11 nodes instead of 10" example.
4. Give a concrete example of "read-your-own-writes" consistency breaking, and one fix.
5. Recite the full database scaling ladder from memory, in order, with the signal that triggers each step.

Next → [Chapter 6: Distributed Systems](#) — the deepest chapter in this roadmap: failures, retries, consensus, idempotency, and the patterns (Saga, Outbox, CQRS, Event Sourcing) that hold distributed systems together.